

VisionGuard AI: A Comparative Medical Image Segmentation and Benchmarking Platform

Anuj Meena

Abstract

This report outlines the architecture and execution of VisionGuard AI, a medical image segmentation system designed to compare classic convolutional approaches (U-Net) against Vision Transformers (SegFormer) and zero-shot foundation models (Segment Anything Model - SAM). The platform provides spatially resolved Grad-CAM feature attention maps and exports optimized execution subgraphs using ONNX Runtime, showing a 2.98x latency speedup during inference. A REST API backend built on FastAPI and an interactive Streamlit dashboard are presented as modern, deployable production interfaces.

I. INTRODUCTION

Medical image segmentation plays a critical role in automated clinical diagnostics. Traditional architectures such as U-Net have relied on convolutional downsampling and skip connections to recover spatial boundaries. However, recent advances in Vision Transformers (ViTs) and promptable foundation models (like SAM) offer alternative paradigms. VisionGuard AI serves as a production-grade benchmark system comparing these distinct segmentation philosophies side-by-side.

II. ARCHITECTURAL SPECIFICATIONS

A. Model Layers

The platform incorporates three distinct model classes:

- **U-Net:** A classic CNN encoder-decoder implementing double convolutions and MaxPool downsampling, paired with transposed convolutions for high-resolution detail preservation.
- **SegFormer:** A hierarchical transformer encoder paired with a lightweight MLP decoder, bypassing the need for positional embeddings.
- **Segment Anything Model (SAM):** A zero-shot promptable foundation network utilizing click coordinates to isolate mask boundaries dynamically.

B. Model Explainability

Grad-CAM (Gradient-weighted Class Activation Mapping) is integrated into the decoders of U-Net and SegFormer. By backpropagating logit activations relative to the final decoder convolutional layers, spatial attention weight matrices are compiled and overlaid as semi-transparent heatmaps.

III. PERFORMANCE BENCHMARKS

Execution latencies were benchmarked on standard single-threaded CPU pipelines (Batch size = 1, Resolution = 256x256).

TABLE I
INFERENCE LATENCY COMPARISONS

Runtime Engine	Device	Average Latency (ms)	Throughput (FPS)
PyTorch FP32	CPU	115.42 ms	8.66
ONNX Runtime (Optimized)	CPU	38.65 ms	25.87

The optimized ONNX Runtime model demonstrates a **2.98x speedup factor** compared to the native PyTorch FP32 baseline.

IV. PRODUCTION ENGINEERING

The system is standardized with production-ready devops tools:

- 1) **FastAPI REST Server:** Exposes prediction, interactive SAM segmentation, and Grad-CAM explainability endpoints.
- 2) **Streamlit Dashboard:** Provides image upload fields, opacity adjustment sliders, and Plotly latency comparison graphs.
- 3) **Containerization:** Standardized via a multi-stage, lightweight Docker container.
- 4) **Static Analysis:** Standardized with Ruff formatting and strict Mypy types.